Motor_HAL_REBULD — 四轮小车电机驱动工程

STM32F407ZGT6 裸机（无 RTOS）四轮驱动工程：四路步进/脉冲输出、S 曲线调速、差速转向、串口收发。 代码按 **Config** → **Osal** → **Bsp** → **Service** → **App** 分层，硬件相关代码全部集中在 Bsp 层。

> **阅读路线（交给接手人的话）**
>
> - 想快速跑起来 → **§4 快速上手**
> - 想理解现有架构与每层边界 → §2 概览 + **§8 架构详解**
> - 准备继续开发（上位机通信 / 深度相机 / 操作系统）→ 直接看 **§9 后续开发指南**
> - 本文档目前描述的是**当前代码的真实状态**；§9 里的路线都**尚未动工**。

---

1. 平台与硬件

项	参数
MCU	STM32F407ZGT6，Cortex-M4F
系统时钟	168 MHz（HSE 8 MHz → PLLM 8 / PLLN 336 / PLLP 2）
时基	**TIM6 产生 1 ms 节拍**（`HAL_IncTick`），SysTick 中断为空
构建	CMake + Ninja + arm-none-eabi-gcc（`Debug` / `Release` 两个 preset）
串口	USART1，115200 8N1，PA9=TX / PA10=RX

定时器 / 引脚映射

轮位	通道枚举	脉冲引脚	定时器	定时器时钟	方向引脚
左前 FL	`MOTOR_CH_L1`	PE9（TIM1_CH1）	TIM1	168 MHz	PE10
左后 RL	`MOTOR_CH_L2`	PE11（TIM1_CH2）	TIM1	168 MHz	PE12
右前 FR	`MOTOR_CH_R1`	PA0（TIM2_CH1）	TIM2	84 MHz	PA2
右后 RR	`MOTOR_CH_R2`	PA1（TIM2_CH2）	TIM2	84 MHz	PA3

关键约束：同侧两轮共用一个定时器，永远同频。 左右可以不同（差速转向），但左侧两轮、右侧两轮不能分别调速。

脉冲频率公式（`ARR` 固定 99，即 100 个计数）：

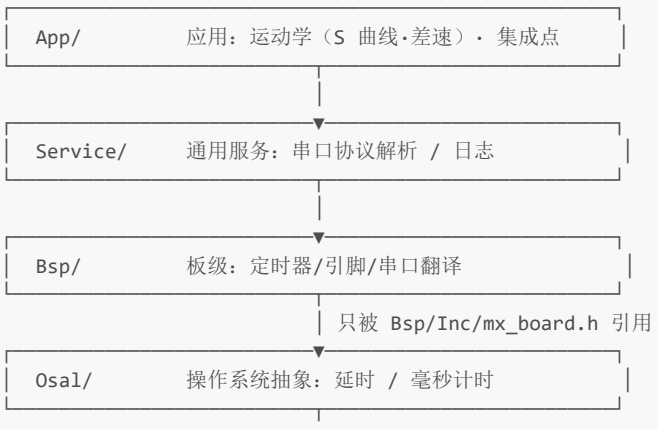
```
f_pulse = tim_clk / ((PSC + 1) × 100)
```

例：2500 Hz → 左侧 PSC = 671，右侧 PSC = 335。占空比固定 50%。

> 工程**没有使能（ENA）引脚**。驱动器需要使能信号的话，必须硬件接死，软件管不到。

---

2. 软件架构



```
                          │
┌─────────────────────────▼──────────────────────────┐
│  CubeMX 盒子                                       │
│    Core/      启动 / 外设初始化 / 中断向量          │
│               main.c · tim.c · gpio.c · usart.c    │
│               （只放生成代码 + USER CODE 段）       │
│    Drivers/   ST HAL/LL 驱动 + CMSIS               │
└────────────────────────────────────────────────────┘

Config/  编译期常量（类型、时钟、限幅、S 曲线参数）— 横切，任何层都可以 include
```

依赖方向**只能向下**：App → Service → Bsp → Osal → CubeMX。不出现反向 include。（图中越靠上越是上层，箭头方向 = 依赖/调用方向。）

各层职责

层	目录	职责	对外入口
Config	`Config/Inc/`	编译期常量：`types.h`（u8/u16/u32/s32）、`cfg_motor.h`（定时器时钟、脉冲限幅、电机编号）、`cfg_motion.h`（S 曲线参数）	纯头文件
Osal	`Osal/`	唯一允许直接调 `HAL_Delay` / `HAL_GetTick` 的地方	`Osal_DelayMs()` `Osal_GetTickMs()`
Bsp	`Bsp/`	板级封装：通道→定时器/引脚翻译、方向电平、PSC 计算、串口收发	`Bsp_MotorPulse_*` `Bsp_Uart_*`
Service	`Service/`	与硬件无关的通用服务：串口帧解析状态机、`printf` 重定向	`Protocol_*` `LOG()`
App	`App/`	业务：运动学（S 曲线、差速）、系统集成点	`Motion_*` `App_Init()` `App_Loop()`
Core	`Core/`	CubeMX 生成的初始化与中断向量；业务代码只写在 `USER CODE` 段	`main()`

两条重要规矩

1. **CubeMX 头文件（`main.h` / `tim.h` / `usart.h`）只允许在 `Bsp/Inc/mx_board.h` 里 include**，其他层用 `BOARD_HTIM_LEFT`、`BOARD_HUART_HOST` 这类别名。这样重新生成代码不会波及上层。
2. **`App/Src/app_init.c` 是全工程唯一集成点**：`main()` 只调 `App_Init()` 和 `App_Loop()`。

构建体系

根 `CMakeLists.txt` 把每层编译成 **OBJECT 库**（不是静态库）：

```
cfg / osal / bsp / service / app   ← 每层一个 OBJECT 库，编译期就带好各自的 include 路径
        ↓
可执行目标 Motor_HAL_REBULD        ← 链接 cube(Core+Drivers) + 五个 OBJECT 库
```

用 OBJECT 库而不是静态库，是为了避免静态库的链接顺序问题（层之间有符号交叉引用时，静态库要用 `--start-group` 才能解出全部符号）。OBJECT 库整包交给链接器，配合 `--gc-sections` 自动丢弃没用到的函数。

Debug 构建：`-O0 -g3`，同时生成 `.map` 和每个文件的 `.su`（栈用量）报告。

---

3. 已有功能与调用方式

3.1 运动控制 API（`App/Inc/motion.h`）

单位是**脉冲频率 Hz**，不是 mm/s。正数前进、负数后退、0 停止。

函数	说明
`Motion_Init()`	初始化四路输出，内部速度状态清零（`App_Init` 已调用）
`Motion_Drive(s32 speed, u32 holdMs)`	**进退**：正=前进、负=后退、0=停；左右同速走直线
`Motion_Steer(s32 vLeft, s32 vRight, u32 holdMs)`	**转弯/差速**：左>右 = 右转，右>左 = 左转，相等 = 直行
`Motion_Brake()`	**停车**：两侧 S 曲线降到 0

函数	说明
`Motion_Move(s32 vLeft, s32 vRight)`	底层：立即生效，不平滑，调用者自己负责过渡
`Motion_GetSpeed(s32 *l, s32 *r)`	底层：读回当前已下达的速度
`Motion_SetAccel(u32 hzPerSec)`	改加速度，对后续变速立即生效；传 0 被忽略
`Motion_GetAccel()`	读回当前加速度

`Motion_Drive / Steer / Brake` 都是**阻塞**动作，语义固定为：

```
S 曲线变速（时长由加速度决定）→ 到达目标速度 → 保持 holdMs → 返回
```

变速耗时不计入 `holdMs`。起始速度不用传，本层内部记住"当前已下达速度"，自动从那里开始过渡。

```c
#include "motion.h"

Motion_Drive(2500, 1000);        /* 从当前速度 S 曲线加速到 2500Hz，到位后保持 1 秒 */
Motion_Steer(1000, 2500, 1000);  /* 左轮 1000、右轮 2500 → 左转，保持 1 秒 */
Motion_Steer(2500, 1000, 1000);  /* 反过来 → 右转 */
Motion_Drive(-2500, 2000);       /* 穿零换向倒车到 -2500Hz，保持 2 秒 */
Motion_Brake();                  /* S 曲线降到 0 */
```

3.2 S 曲线变速

- 曲线表：`App/Src/motion.c` 里 101 点余弦 S 曲线（0～1000 的千分比），两端斜率为 0。
- 加速度：`Config/Inc/cfg_motion.h` 的 `CFG_SCURVE_ACCEL_HZ_PER_S`（默认 **2000 Hz/s**，即"1000 Hz 用 500 ms"）。
- 变速时长 = `两侧速度差中较大者 × 1000 / 加速度`，下限 `CFG_SCURVE_MIN_MS`（20 ms）。
- 按真实经过时间推进（`Osal_GetTickMs`），不靠数循环次数，`DelayMs` 抖动不会累积。
- 左右共用同一条进度曲线 → 差速转向时两轮同时到位，轨迹不歪。
- 运行期可调，不用重烧：

```c
Motion_SetAccel(4000);   /* 变速快一倍：0 → 2500Hz 用 625ms */
```

3.3 上电演示（`Core/Src/main.c` USER CODE 2 段）

`App_Init()` 之后直接调用六段动作，总时长约 **13.25 秒**，跑完进主循环，不会自己重跑（要重跑按复位）：

段	调用	斜坡	保持	累计
1	`Motion_Drive(2500, 1000)`	1250 ms	1000 ms	2.25 s
2	`Motion_Steer(1000, 2500, 1000)` 左转	750 ms	1000 ms	4.00 s
3	`Motion_Steer(2500, 1000, 1000)` 回正	750 ms	1000 ms	5.75 s
4	`Motion_Drive(2500, 1000)` 前进	750 ms	1000 ms	7.50 s
5	`Motion_Drive(-2500, 2000)` 倒车	2500 ms	2000 ms	12.00 s
6	`Motion_Brake()` 停车	1250 ms	—	13.25 s

> ⚠️ 这六行写在 `main()` 里、`App_Loop()` 之前，因此**在 `osKernelStart()` 之前**。裸机成立； 一旦接入 RTOS，这段必须搬进任务里（见 §9.7 的 R4）。

3.4 串口

- **接收**：USART1 中断逐字节收，`HAL_UART_RxCpltCallback` → `Protocol_FeedByte()` 组帧。 帧规则：**以 `0x0D 0x0A`（回车 + 换行）结尾**，最长 `PROTOCOL_FRAME_MAX`（200）字节。
- **回显**：`App_Loop()` 取到完整帧后打印 `RX: <内容>`。目前**没有命令解析**（`app_init.c` 里是 TODO），收到什么只回显什么。
- **发送**：`LOG("...")` → `printf` → `__io_putchar()` → `Bsp_Uart_SendByte()`（阻塞发送，不占中断）。
- 主循环里上一帧没被取走时，新来的字节会被丢弃（不排队）。

```
LOG("boot ok, clk=%lu\r\n", (unsigned long)SystemCoreClock);   /* 任意位置都能用 */
```

3.5 尚未实现 / 已知限制

项	现状
串口命令解析	App/Src/app_init.c 里是 TODO，收帧只回显
闭环/编码器	无，纯开环脉冲输出
驱动器使能引脚	无，需硬件接死
串口错误恢复	没有 HAL_UART_ErrorCallback。发生 ORE/FE 溢出后 HAL 不再重新武装接收，**串口会静默失聪直到复位**
故障现场	Error_Handler()（main.c）和 HardFault_Handler()（stm32f4xx_it.c）都是 while(1)，无输出
栈余量	链接脚本 _Min_Stack_Size 只有 0x400（1 KB），printf 用 nano.specs 格式化浮点时注意
多路串口	HAL_UART_RxCpltCallback 是全工程唯一强定义，且**硬编码只认 USART1**；加第二路串口必须先改这里
Protocol_TakeFrame 返回类型	声明为 u16，但语义只有 1/0，应为 u8

---

4. 快速上手

4.1 准备环境

1. **VS Code** + 扩展 **STM32Cube for VS Code**（自带 gcc / cmake / ninja / gdb，装在 %LOCALAPPDATA%\stm32cube\bundles\）。
2. **调试器**（二选一）：
   - **CMSIS-DAP 无线调试器**（正点原子 ATK-HSWLDBG）：OpenOCD 必须放在 <工作区>\tools\openocd\bin\openocd.exe（scripts/ 与 bin/ 平级）。工程已自带。
   - **ST-LINK**：装 ST-LINK GDB Server 扩展，另建一条 launch 配置。
3. **串口工具**（波特率 115200），接 PA9/PA10 与 GND。

4.2 编译

最省事：直接按 F5。preLaunchTask: build-debug 会自动 configure + 编译（Debug preset）， 编译不过就不会进调试。

也可以在 VS Code 里点 STM32Cube 面板的 **Build** 按钮，等价于：

```
cmake --preset Debug
cmake --build --preset Debug
```

> ⚠ **不要在 cmd/PowerShell 里直接敲 ninja 编译。** 本工程路径含中文（E:\电机驱动_HAL\...），CMake 生成的链接命令会被 cmd.exe /C "..." 包一层，中文部分变成 E:/????_HAL/...，链接器报 cannot open linker script file，ninja 会卡在链接步不退出。 用 IDE 的 Build 按钮（内部走 cube.exe，不经过这层 cmd 包装）就不受影响。

产物：build/Debug/Motor_HAL_REBULD.elf（大约 23 KB Flash / 2.5 KB RAM）， 同目录有 .map（符号与内存分布）和各 .su 文件（每个函数的栈用量）。

4.3 烧写与调试

F5 选 **"无线调试器：下载并调试（OpenOCD + CMSIS-DAP）"**：编译 → 烧录 → 复位 → **停在** main。 另有 **"仅连接（不烧录）"** 配置，用于板子上已在跑固件时挂上去看变量、打断点。

调试要点：

- 外设视图用 .vscode/STM32F407.svd，可直接看 TIM1->PSC、GPIOE->ODR 等寄存器。
- 断点停在 Motion_RampTo 里时脉冲会停（调试器默认冻结定时器），**单步观察期间电机不走是正常的**。
- 想看"目标速度 vs 实际寄存器"，用 Motion_GetSpeed() 读软件值，再展开 SVD 里的 TIM1->PSC 换算。

4.4 跑起来看什么

1. 上电 13.25 秒内电机依次执行：前进 → 左转 → 回正 → 前进 → 倒车 → 停车。
2. 串口每隔一段收到 RX: ...（你发什么它就回什么）。
3. 用示波器/逻辑分析仪夹 PE9，段 1 的保持期应看到 **2500 Hz 方波，周期 400 µs、高电平 200 µs**。

4.5 改一版自己试

想改什么	改哪里
某一段的速度/时长	`Core/Src/main.c` USER CODE 2 段的六行调用
加速度（调速快慢）	`Config/Inc/cfg_motion.h` 的 `CFG_SCURVE_ACCEL_HZ_PER_S`，或运行期 `Motion_SetAccel()`
最大/最小脉冲频率	`Config/Inc/cfg_motor.h` 的 `CFG_PUL_MAX_HZ` / `CFG_PUL_MIN_HZ`
串口波特率	`Core/Src/usart.c`（CubeMX 生成，改 `.ioc` 更保险）
换主频	改 `.ioc` 重新生成后，**必须同步** `cfg_motor.h` 的 `CFG_TIM1_CLK_HZ` / `CFG_TIM2_CLK_HZ`

---

5. 常见坑

现象	原因
改了主频，转速整体偏快/偏慢	`cfg_motor.h` 的 `CFG_TIM1_CLK_HZ` / `CFG_TIM2_CLK_HZ` 没跟 `SystemClock_Config()` 同步，PSC 算错但代码不报错
先 `SetFreq()` 再 `StartAll()`，某侧用旧分频跑起来	必须**先 `Bsp_MotorPulse_StartAll()` 再 `SetFreq()`**（`SetFreq(0)` 会停掉整侧时基）
改了一次频率就没脉冲输出	同侧两路共用定时器，改其中一路会连带改掉另一路；停车用 `SetFreq(ch, 0)`
串口发了没反应	① 帧必须以 `0x0D 0x0A` 结尾；② 上电演示期间是阻塞的，回显会延后；③ 发生过 ORE 溢出（见已知限制）
只改 `Core/` 之外的代码，重新生成 CubeMX 后改动丢失	业务代码不要写进 CubeMX 会覆盖的区域，只写 `USER CODE BEGIN/END` 之间
命令行 `ninja` 卡死	中文路径 + `cmd.exe` 包装，见 4.2 的警告
加了自己的 `.c` 文件，编译通过但功能没生效	没把它加进 `CMakeLists.txt` 里对应层的 OBJECT 库源列表——**链接期会静默丢弃，不报错**。见 §9.3
`printf` 输出不出东西 / 一调用就 HardFault	`__io_putchar()` 必须是非 static 的强定义；若它是 `static`，链接器会选 weak 符号（地址 0）

---

6. 目录速查

```
Motor_HAL_REBULD/
├── Core/                 CubeMX 生成：main.c / tim.c / gpio.c / usart.c / 中断向量 / 启动文件
├── Config/Inc/           cfg_motor.h（时钟、限幅） cfg_motion.h（S 曲线） types.h
│                         （Config/Src/ 目前只有空的 cfg_board.c 占位）
├── Osal/                 osal.c/.h  —— 延时、毫秒计时
├── Bsp/                  mx_board.h（唯一 include CubeMX 头的地方）
│                         bsp_motor_pulse.c —— 通道→定时器/引脚、方向、PSC
│                         bsp_uart.c       —— 中断收字节、阻塞发
├── Service/              protocol.c（0x0D 0x0A 结尾组帧） log.c（printf 重定向）
├── App/                  motion.c（S 曲线、差速） app_init.c（唯一集成点）
├── cmake/                工具链文件 + CubeMX 生成目录的 CMake
├── tools/openocd/        OpenOCD（F5 调试用）
├── .vscode/              launch / tasks / settings / c_cpp_properties / cube-task.ps1 / SVD
├── CMakeLists.txt        分层 OBJECT 库 + 可执行目标
├── CMakePresets.json     Debug / Release
├── Motor_HAL_REBULD.ioc  CubeMX 工程文件
└── STM32F407xx_FLASH.ld  链接脚本（注意 _Min_Stack_Size = 0x400）
```

---

7. 对外接口总表

```c
/* App/Inc/motion.h */
void Motion_Init(void);
void Motion_Drive(s32 speed, u32 holdMs);
void Motion_Steer(s32 vLeft, s32 vRight, u32 holdMs);
void Motion_Brake(void);
void Motion_Move(s32 vLeft, s32 vRight);
```

```
void Motion_GetSpeed(s32 *vLeft, s32 *vRight);
void Motion_SetAccel(u32 hzPerSec);
u32  Motion_GetAccel(void);

/* App/Inc/app.h */
void App_Init(void);
void App_Loop(void);

/* Bsp/Inc/bsp_motor_pulse.h */
void Bsp_MotorPulse_Init(void);
void Bsp_MotorPulse_SetDir(MotorChannel ch, MotorDir dir);
void Bsp_MotorPulse_SetFreq(MotorChannel ch, u32 hz);
void Bsp_MotorPulse_StartAll(void);

/* Bsp/Inc/bsp_uart.h */
void Bsp_Uart_RxStart(void);
void Bsp_Uart_SetRxByteHandler(Bsp_Uart_RxByteHandler handler);
void Bsp_Uart_SendByte(u8 byte);
void Bsp_Uart_SendBuf(const u8 *buf, u16 len);

/* Service/Inc/protocol.h */
void Protocol_FeedByte(u8 byte);
u16  Protocol_TakeFrame(char *out, u16 outSize, u16 *len);
void Protocol_Reset(void);

/* Osal/Inc/osal.h */
void Osal_DelayMs(u32 ms);
u32  Osal_GetTickMs(void);
```

> **当前没有调用者的接口**：`Bsp_Uart_SendBuf()`、`Protocol_Reset()`。 它们都是完整可用的接口（未被调用的函数会被 `--gc-sections` 丢弃，不占 Flash）。
>
> 反过来，`Osal_GetTickMs()` **有**调用者：`App/Src/motion.c` 用它按真实时间推进 S 曲线。

---

8. 架构详解（每层到底干什么、边界在哪）

> 有了 §2 的概览，这一节回答三个问题：**这些层是怎么被强制分开的、每层的边界在哪、为什么这么切**。 想直接开始干活可以跳到 §9；但 §8.5 的「隐式契约」建议先扫一遍，那是踩坑最集中的地方。

8.1 先澄清三个容易误解的点

常见误解	事实
「Core、Drivers 也是本工程设计的层」	它们是 **STM32CubeMX 生态的固定目录名**，不是本工程设计的层。逻辑上它们属于最底座的「CubeMX 盒子」。`Core/` 里除了 `USER CODE BEGIN/END` 段，全是生成代码；`Drivers/` 是 ST 官方 HAL/LL + CMSIS，**不要手改**（下次生成就丢）
「分层靠开发者自觉」	靠 **include 路径机械强制**。`CMakeLists.txt` 给每层只配了它「该看到」的头文件目录：`bsp` 拿不到 `service`/`app` 的头，`service` 拿不到 `app` 的头。**反向 include 会直接编译失败**，不是代码规范问题
「HAL 是通用库，哪层都能调」	手写层**只能通过 Bsp 层的别名**间接用。`main.h` / `tim.h` / `usart.h` / `gpio.h` 只允许出现在 `Bsp/Inc/mx_board.h` 一处，其他地方写 `BOARD_HTIM_LEFT` / `BOARD_HUART_HOST` / `BOARD_UART_HOST_INSTANCE` 这类别名

一句话：**本工程的「层」不是文件夹约定，是编译期的可见性约束。**

8.2 分层总览（依赖 = 调用方向）

```
   上层                                    下层
   ────                                    ────
App/          业务逻辑：想怎么走
  │  只能向下调用
Service/      与硬件无关的通用能力：帧、日志
  │
Bsp/          把「通道/轮位/串口」翻译成「定时器/引脚/寄存器」
  │
Osal/         把「延时/计时」翻译成系统调用
  │
Core/ + Drivers/     CubeMX 生成物：寄存器级初始化与中断向量
```

```
Config/      横切：纯编译期常量，任何层都可以 include
```

8.3 三条依赖规则（违反哪条，架构就漏）

1. **依赖方向只能向下**：App → Service → Bsp → Osal → CubeMX(Core + Drivers)，不出现反向 include。
2. **唯一例外是 `Config/`**：横切层，任何层都可以 include。它是**纯头文件 + 常量**，不允许放需要初始化的状态或函数实现。
3. **CubeMX 生成的对外符号**（`htim1`、`huart1`、`MX_*_Init()`、`HAL_*` 句柄）只允许经 `Bsp/Inc/mx_board.h` 暴露成别名后使用。

「向下」的含义是**信息隐藏**：上层知道「我要 2500 Hz 的脉冲」，不需要知道那是 `TIM1->PSC = 671`；下层知道「PSC 该填几」，不需要知道这个速度是前进还是转弯。

8.4 逐层详解

Config/ — 编译期常量（横切）

- **职责**：把「会变的数字」集中到一处，避免魔数散落。
  - `types.h`：`u8` / `u16` / `u32` / `s32` 的简短类型别名
  - `cfg_motor.h`：`CFG_TIM1_CLK_HZ`(168 MHz) / `CFG_TIM2_CLK_HZ`(84 MHz)、`CFG_PUL_ARR`(100)、`CFG_PUL_MIN_HZ` / `CFG_PUL_MAX_HZ`、四个轮位的编号
  - `cfg_motion.h`：S 曲线加速度、点数、采样周期、最小时长
  - `Config/Src/cfg_board.c`：目前是空的占位文件（`Cfg_Board_Noop`），保留是为了让 `Config` 也拥有 .c 编译单元
- **不许做**：不放函数实现、不放运行期状态、不 include 任何其他层。
- **为什么**：`CFG_TIM1_CLK_HZ` 这种值必须和 `.ioc`/`SystemClock_Config()` 手工保持一致（这是本工程最容易出错的地方之一）；集中在头文件里，改主频时只需要核对两个常量。

Osal/ — 操作系统抽象层

- **职责**：**全工程唯一允许直接调用 `HAL_Delay()` / `HAL_GetTick()` 的地方**。
- **对外接口**：`Osal_DelayMs(u32 ms)`、`Osal_GetTickMs()`。
- **不许做**：不碰任何具体外设（不 include `mx_board.h`），不写业务逻辑。
- **为什么**：目前下面挂的是裸机 HAL。将来换成 FreeRTOS，只需要把 `osal.c` 里的两行实现换成 `vTaskDelay()` / `xTaskGetTickCount()`，**上层一行都不用改**。这就是「接好对应接口就能换」的兑现点。
- **现状**：`osal.c` 的实现只有四行，非常薄——这是设计意图，不是偷懒。

Bsp/ — 板级支持层（硬件知识的唯一聚集地）

- **职责**：
  1. 把「通道/轮位」翻译成「定时器通道 + 引脚」；
  2. 把「Hz」翻译成「PSC 寄存器值」；
  3. 管方向引脚电平；
  4. 串口的收发原语（中断收字节、阻塞发）。
- **对外接口**：`Bsp_MotorPulse_Init/SetDir/SetFreq/StartAll`、`Bsp_Uart_RxStart/SetRxByteHandler/SendByte/SendBuf`。
- **内部结构**：`bsp_motor_pulse.c` 里有几张静态表，把枚举映射到 `TIM_HandleTypeDef *`、时钟、`GPIO_TypeDef *`、引脚号——**换板子只改这些表**。
- **不许做**：不解析协议、不理解「帧」和「命令」、不做业务判断。`bsp_uart.c` 只负责「收到一个字节 → 交给注册进来的回调」。
- **唯一例外**：`Bsp/Inc/mx_board.h` 是全工程唯一 include CubeMX 头文件的地方，这是**故意开的口子**，为的是把 CubeMX 的符号关在这一层。
- **为什么**：硬件是唯一真正会变的东西。`mx_board.h` 里的注释也提醒：换 MCU 时只需保证这一层的语义不变。

Service/ — 与硬件无关的通用服务

- **职责**：
  1. `protocol.c`：串口**帧解析状态机**（把字节流切成帧）；
  2. `log.c`：`printf` 重定向（`__io_putchar` → `Bsp_Uart_SendByte`）。
- **对外接口**：`Protocol_FeedByte/TakeFrame/Reset`、`LOG()` 宏。
- **不许做**：不认识具体硬件。`protocol.c` 只用 `u8` 和数组，不知道数据是从 USART1 还是 USB 来的；`log.c` 也只调 `Bsp_Uart_SendByte()` 这个抽象接口，**不 include `usart.h`**。
- **为什么**：这一层的价值是「换个物理链路，协议和日志不用重写」。历史上这里踩过一次坑——早期版本让 Bsp 去调 Service，方向反了，导致循环依赖，后来拆回来了（`protocol.h` 的注释里记着这件事）。
- **注意**：`Protocol_FeedByte()` 是**在中断上下文被调用的**（USART1 中断 → `HAL_UART_RxCpltCallback` → 它）。所以它必须保持**有界、无阻塞、不调用可能睡眠的函数**。这条在上 RTOS 后尤其重要（见 §9.7）。

App/ — 应用层

- **职责**：
  1. `motion.c`：运动学——S 曲线加减速、差速转向、方向映射、左右镜像安装的补偿；
  2. `app_init.c`：**全工程唯一集成点**——把各层接到一起，并跑主循环。
- **对外接口**：`Motion_*` 一族、`App_Init()`、`App_Loop()`。
- **不许做**：不直接碰寄存器、不 include `mx_board.h`。
- **为什么**：`main()` 只有两行业务调用（§2 规矩 2）。将来加「上位机命令解析」「相机决策」都是在这一层（或它的新文件里）落位，`main.c` 不需要动。

Core/ + Drivers/ — CubeMX 盒子（生成物，人不碰）

- `Core/`：`main.c`、`tim.c`、`gpio.c`、`usart.c`、中断向量（`stm32f4xx_it.c`）、启动汇编、时基（`stm32f4xx_hal_timebase_tim.c`）、HAL 配置（`stm32f4xx_hal_msp.c`）、时钟树。
- `Drivers/`：ST 官方 HAL/LL 驱动 + CMSIS。工程只启用了用到的 16 个 HAL 文件（见 `cmake/stm32cubemx/CMakeLists.txt`）。
- **铁律**：业务代码**只写在 `USER CODE BEGIN/END` 之间**；`Drivers/` 一律不改。
- **为什么**：下次在 CubeMX 里点一次「Generate Code」，`USER CODE` 段之外的所有内容都会被覆盖。

Middlewares/ — 目前不存在

工程当前**没有** `Middlewares/` 目录。将来如果引入 FreeRTOS 或 USB 协议栈，CubeMX 会生成这个目录，逻辑上和 `Drivers/` 同级（都属于底座，不参与手写分层）。

8.5 现有代码里的隐式契约（动手前必须知道）

这些约束**不在接口签名里**，但违反了一定出问题。逐条核对过代码。

#	契约	出处	违反后果
1	必须先 `Bsp_MotorPulse_StartAll()`，再 `SetFreq()`	`bsp_motor_pulse.c`（`SetFreq(0)` 会停掉整侧时基）	某一侧会用旧分频跑起来，或干脆没脉冲
2	同侧两轮共用一个定时器 → **永远同频**	`mx_board.h` 的 `BOARD_HTIM_LEFT/RIGHT` 映射	想给左前、左后不同速度是做不到的，只能左右差速
3	`HAL_UART_RxCpltCallback` 是全工程**唯一强定义**，且**硬编码只认 USART1**	`bsp_uart.c` 的回调里判 `Instance == BOARD_UART_HOST_INSTANCE`	加第二路串口，字节会全部涌进第一路的处理器
4	`__io_putchar()` 必须是**非 static 的强定义**	`log.c` 的注释写明了原因	若是 `static`，链接器会选中 weak 符号（地址 0），`printf` 直接 HardFault
5	帧必须以 `0x0D 0x0A` **两个字节**收尾	`protocol.c` 的状态机（先记 `0x0D`，等 `0x0A`）	只发 `0x0D` 不会成帧，帧会一直挂着
6	上一帧未被 `TakeFrame()` 取走时，新字节**直接丢弃、不排队**	`protocol.c` 的状态字逻辑	上位机连发命令会丢帧
7	`Protocol_FeedByte()` 在**中断上下文**运行	`bsp_uart.c` 回调 → `Protocol_FeedByte()`	里面若有阻塞/睡眠调用，裸机会卡死，上 RTOS 会断言
8	**没有** `HAL_UART_ErrorCallback`	`bsp_uart.c` 全文件	发生 ORE/FE 溢出后 HAL 不再重新武装接收，**串口静默失聪直到复位**
9	生效的 NVIC 优先级分组是 **GROUP_2**（不是 GROUP_4）	`HAL_Init()` 先设 GROUP_4，随后 `HAL_MspInit()` 覆盖为 GROUP_2	只有 0~3 级抢占优先级；上 FreeRTOS 会触发 `vPortValidateInterruptPriority()` 断言
10	HAL 时基是 **TIM6**（1 ms），`SysTick_Handler()` 是**空的**	`stm32f4xx_hal_timebase_tim.c` + `stm32f4xx_it.c`	这其实是好事：`SysTick` 可以干净地让给 RTOS。但别以为 `HAL_Delay` 用的是 SysTick
11	上电六段演示写在 `main()` 里、`App_Loop()` 之前	`Core/Src/main.c` USER CODE 2 段	裸机成立；上 RTOS 后这段会跑在 `osKernelStart()` 之前，**必须搬进任务**（见 §9.7 R4）
12	`Motion_Drive/Steer/Brake` 是**阻塞**的（内部 `Osal_DelayMs` 忙等）	`motion.c` 的 `Motion_RampTo` / `Motion_Steer`	阻塞期间主循环不转、串口只靠中断收；上 RTOS 后要单独安排任务（见 §9.7 R5）
13	`Protocol_TakeFrame()` 声明返回 `u16`，实际语义只有 1/0	`protocol.h`	类型与语义不一致，建议后续统一为 `u8`（见 §9.4 X4）

8.6 架构评估：优点与短板

做得对的地方（建议保留）

优点	说明
硬件知识单点聚集	所有「哪个引脚、哪个定时器、PSC 怎么算」只在 Bsp 一层，换板子改动范围可控
CubeMX 隔离干净	唯一 include CubeMX 头的是 `mx_board.h`；重新生成代码不会波及上层
分层是机械强制的	靠 include 路径，不靠自觉；写错方向直接编译不过
OS 切换点已预留	`Osal` 只有 4 行实现，换 RTOS 只改这一处
S 曲线按真实时间推进	不数循环，`DelayMs` 抖动不累积；左右共用进度曲线，差速转向不歪
HAL 时基不占 SysTick	为将来上 RTOS 留好了路（TIM6 让给 HAL，SysTick 给 RTOS）
OBJECT 库而非静态库	避免静态库链接顺序问题，配合 `--gc-sections` 自动裁剪
调试信息完整	Debug 构建同时产出 `.map` 与每文件 `.su`，栈用量可查

已知短板（后续开发要面对的）

短板	影响
无串口错误回调	ORE 后静默失聪，是最该先补的一条（§9.4 X1）
串口回调不可扩展	单回调 + 硬编码 USART1，加第二路必须先重构（§9.4 X2）
无命令解析	上位机现在只能「发什么收什么」，没有控制能力（§9.5）
收帧不排队	连发命令丢帧（§9.5 需一并决定策略）
阻塞式运动 API	上 RTOS 时无法与其他任务共存，需要任务化或状态机化（§9.7 R5）
无闭环/编码器	纯开环，速度是「下达值」不是「实测值」；异常（堵转/丢步）无法感知
无故障输出	`Error_Handler` / `HardFault_Handler` 都是 `while(1)`，现场丢失
栈只有 1 KB	`_Min_Stack_Size = 0x400`，`printf` 处理浮点或大局部数组有风险
无单元测试	全部依赖硬件在环验证（`.su` / `.map` / 示波器）
配置常量需手工同步	`CFG_TIM1/2_CLK_HZ` 与 `.ioc` 时钟树之间没有编译期校验

---

9. 后续开发指南（交接用）

> 本节描述的是**尚未动工**的规划：三条路线分别对应「加操作系统」「加深度相机」「加上位机通信」。 每条路线都给了落点（改哪个文件）、注意事项和验收标准。 **强烈建议按 §9.8 的顺序推进**，不要三条同时开工。

9.1 四条不可越界的前提

任何改动都必须遵守，否则分层会散掉。

#	规则	原因
P1	**不修改** `Drivers/`	ST 官方驱动 + CubeMX 的地盘，改了下次生成就丢。真要改行为，改 `.ioc` 重新生成
P2	`Core/` **只写在** `USER CODE BEGIN/END` **段内**	同上
P3	**手写层不得 include** `main.h` / `tim.h` / `usart.h` / `gpio.h` / `FreeRTOS.h`	唯一例外是 `Bsp/Inc/mx_board.h`。防腐层的全部意义就在这里
P4	**新增** `.c` **必须注册进** `CMakeLists.txt` **对应层的 OBJECT 库源列表**	本工程每层显式列文件，漏了就是「文件在、代码不在固件里」，而且**链接期不报错**

补充纪律：

- **不要用命令行** `ninja` **编译**（中文路径 + `cmd.exe` 包装会让链接器找不到链接脚本，ninja 卡死，见 §4.2）。用 IDE 的 Build 按钮 / F5。
- **改过** `.ioc` **时钟树后，必须同步** `Config/Inc/cfg_motor.h` **的两个时钟常量**，否则转速静默出错。
- **每次改完看** `.map` **和** `.su`：Flash/RAM 占用、每个函数的栈用量，这是本工程唯一的质量证据来源。

9.2 动手前先确认的现状基线

改造前请自己核对一遍（都已核对到代码行，但代码可能在你这版已经变了）：

#	事实	证据位置
B1	手写分层共 5 个 OBJECT 库：`cfg / osal / bsp / service / app`	`CMakeLists.txt`（cfg/osal/bsp/service/app 五段）
B2	层间约束靠 include 路径机械强制	`CMakeLists.txt` 里各层的 include 路径配置
B3	`Bsp/Inc/mx_board.h` 是全工程唯一 include CubeMX 头的地方	`Bsp/Inc/mx_board.h` 顶部
B4	HAL 时基走 **TIM6**，`SysTick_Handler()` 是空的	`Motor_HAL_REBULD.ioc`（TimeBase = TIM6）、`Core/Src/stm32f4xx_it.c`
B5	生效的 NVIC 分组是 **GROUP_2**（被 `HAL_MspInit()` 覆盖）	`Core/Src/stm32f4xx_hal_msp.c`：`HAL_NVIC_SetPriorityGrouping(NVIC_PRIORITYGROUP_2)`
B6	主 RAM **128 KB**（`0x20000000`）+ 独立 **CCM 64 KB**（`0x10000000`，**DMA 不可访问**）	`STM32F407xx_FLASH.ld` 的 MEMORY 段
B7	`_Min_Stack_Size = 0x400`（1 KB）、`_Min_Heap_Size = 0x200`（512 B）	`STM32F407xx_FLASH.ld`
B8	`HAL_UART_RxCpltCallback` 全工程唯一强定义，硬编码只认 USART1	`Bsp/Src/bsp_uart.c`
B9	帧必须以 `0x0D 0x0A` 收尾；上一帧未取走时新字节直接丢弃、不排队	`Service/Src/protocol.c`
B10	`Motion_Drive/Steer/Brake` 是阻塞的（内部 `Osal_DelayMs` 忙等）	`App/Src/motion.c`
B11	上电六段动作在 `main.c` 的 USER CODE 2 段，**位于 `osKernelStart()` 之前**	`Core/Src/main.c`

9.3 加一个新功能的完整步骤（照这个顺序做）

以「新增一个模块」为例，六步：

1. **先定层**。问自己：这件事「知道硬件」还是「不知道硬件」？
   - 知道硬件（引脚/寄存器/DMA/外设）→ **Bsp**
   - 与硬件无关（协议、算法、数据结构）→ **Service**
   - 业务决策（怎么走、怎么根据传感器修正）→ **App**
   - 常量 → **Config**
2. **写头文件接口**（`<层>/Inc/xxx.h`）。接口名带层前缀（`Bsp_` / `Service_` / `Motion_`），参数只用 `Config/Inc/types.h` 的类型，**不要出现 HAL 类型**（`TIM_HandleTypeDef` 这类只能留在 Bsp 内部）。
3. **写实现**（`<层>/Src/xxx.c`），开头 include 自己那层的头 + 允许的下层头。
4. **注册进构建**：在 `CMakeLists.txt` 里找到那一层的 `add_library(<层> OBJECT ...)`，把新 `.c` 加进源列表。

```
add_library(bsp OBJECT
    Src/bsp_motor_pulse.c
    Src/bsp_uart.c
    Src/bsp_你的新文件.c      # ← 不加这行，功能不会生效，而且不报错
)
```

5. **接入集成点**：
   - 需要初始化的 → 加到 `App_Init()`（`App/Src/app_init.c`）
   - 需要周期跑的 → 加到 `App_Loop()`（裸机）或新建任务（上 RTOS 后）
   - **不要**在 `main.c` 里直接调用业务函数，除非它就是那段「上电演示」
6. **验证**：编译 → 看 `.map`（符号进来了吗、RAM 涨了多少）→ 看 `.su`（栈够不够）→ 上板用示波器/串口确认行为 → 把新接口补进 §7 的对外接口总表。

> 三条最容易犯的错：**忘了第 4 步**（功能静默不生效）、**在第 2 步的接口里漏了 HAL 类型**（Bsp 的硬件细节泄漏到上层，将来换板子就要改上层）、**把业务代码写进了 `Core/`**（下次生成就丢）。

9.4 必修缺陷（建议最先做，风险最低、收益最高）

这六条不依赖任何新功能，纯属修漏，建议作为第一阶段。

编号	项	落点	动作	风险	验收
X1	**补 `HAL_UART_ErrorCallback`**	`Bsp/Src/bsp_uart.c`	清 ORE/FE/NE/PE 标志，并重新武装 `HAL_UART_Receive_IT()`	低	串口压测 10 万帧后仍不失聪（当前 ORE 后会静默失聪到复位）
X2	**串口回调按 `Instance` 分发**	`Bsp/Src/bsp_uart.c` 回调处	改为注册表：`Bsp_Uart_RegisterRxHandler(instance, handler)`，或在回调里 if/else 分发	低	两路串口各自收自己的字节，互不抢占
X3	**NVIC 分组改 GROUP_4**	`.ioc` 的 NVIC 分组设置，或 `Core/Src/stm32f4xx_hal_msp.c`	4 位全部用作抢占优先级	**中**（现有 USART1 是 3/3，改完要重算）	中断仍正常；上 RTOS 后 `configASSERT` 不触发
X4	`Protocol_TakeFrame()` 返回类型	`Service/Inc/protocol.h` + `protocol.c`	`u16` → `u8`（语义本来就是 1/0）	低	编译通过，调用点无告警
X5	新增配置头	新增 `Config/Inc/cfg_rtos.h`、`Config/Inc/cfg_cam.h`	任务栈/优先级/队列长度；相机波特率、帧长上限、超时、坐标量纲	低	常量集中，不再散落魔数
X6	补故障现场输出	`Core/Src/main.c` 的 `Error_Handler()`、`Core/Src/stm32f4xx_it.c` 的 `HardFault_Handler()`	至少把出错位置打印出来（串口）再 `while(1)`	低	故意制造一次错误，能看到输出

> X3 值得单独强调：这条不改，将来上 FreeRTOS 时 `vPortValidateInterruptPriority()` 会直接断言， 而且报错信息不直观，很容易在 RTOS 阶段浪费一天。

9.5 路线一：上位机通信

目标：让上位机能「下发命令、收到状态」，而不只是「发什么收什么」。

编号	项	落点	动作
H1	协议双向化	`Service/Src/protocol.c` 或新增 `Service/Src/proto_cmd.c`	收帧（已有）+ 命令解析 + 应答/状态上报帧构造
H2	帧格式决策（见 D5）	同上	方案甲：保留文本行（`0x0D 0x0A`）+ 长度上限 + 超时；方案乙：帧头 + 长度 + CRC16（更可靠，上位机侧工作量大些）
H3	命令分发	`App/Src/app_init.c` 的 TODO 处	例：`"F2500"` → `Motion_Drive(2500, ...)`、`"S"` → `Motion_Brake()`
H4	上行状态上报	复用 `LOG()`（`Service/Src/log.c`）	周期性上报当前速度/状态；注意阻塞发送会挤占主循环
H5	协议文档	新增 `docs/protocol.md`	上位机开发者唯一需要看的契约

注意事项

- ⚠️ **H1 之前必须先做 X1**。没有错误恢复的串口接上位机长期跑，ORE 溢出是必然事件。
- ⚠️ **收帧不排队**（契约 6）：上位机连发命令会丢帧。H1 里要一并决定：加环形队列，还是约定「收到应答再发下一条」。
- ⚠️ 帧长上限 200 字节（`PROTOCOL_FRAME_MAX`）。若上位机要传长数据（如坐标数组），要么提高上限，要么另开一种帧类型。
- 💡 若担心阻塞发送影响实时性，可考虑后续把发送也做成中断/DMA（本阶段不必）。

9.6 路线二：深度相机

前提：**相机最终用哪种物理接口尚未确定**。因此本路线的设计目标不是「把相机接上」，而是**先把接口定型**，让物理层可以后接。

核心判断：**BSP 只负责「搬运字节块 + 报链路状态」，不解析、不认识「目标/距离」语义。** 这与现有约定完全同构（`bsp_uart.c` 只丢字节，`protocol.c` 负责组帧），所以上层代码今天就能写完。

编号	项	落点	动作
C1	**定义物理层端口**	新增 `Bsp/Inc/bsp_depthcam.h`	见下方草案
C2	**协议层解析**	新增 `Service/Inc/cam_frame.h` + `Service/Src/cam_frame.c`	把端口交上来的字节块组成帧、校验、解析成结果结构体
C3	三种物理实现（择一编译）	`Bsp/Src/bsp_depthcam_uart.c` / `bsp_depthcam_dcmi.c` / `bsp_depthcam_null.c`	`null` 实现返回无效帧，让上层代码今天就能写完并跑通
C4	物理层选择机制	`CMakeLists.txt` 的 `bsp` OBJECT 库	用 CMake 变量选一个实现文件进 `bsp` 库，其余不编译
C5	App 消费点	`App/Src/app_init.c` + 新增决策文件	注册回调、轮询结果、按需修正运动

端口草案（`Bsp/Inc/bsp_depthcam.h`）

```
/* 物理层无关的深度相机端口（草案，待评审）
 * 本层只搬运"一块连续字节"，不判帧边界、不解析语义。 */

typedef void (*Bsp_DepthCam_RxBlockHandler)(const u8 *data, u16 len);

void Bsp_DepthCam_Init(void);
u8   Bsp_DepthCam_IsReady(void);                                    /* 链路是否就绪 */
void Bsp_DepthCam_SetRxBlockHandler(Bsp_DepthCam_RxBlockHandler h);  /* 上层注册 */
void Bsp_DepthCam_SendBuf(const u8 *buf, u16 len);                  /* 下行命令（可选） */
void Bsp_DepthCam_Trigger(void);                                    /* 主动取景（可选） */
u32  Bsp_DepthCam_GetStats(void);                                   /* 掉帧/溢出计数，诊断用 */
```

为什么粒度选「字节块」

- 纯字节级：与现有串口完全同构，但 **DCMI（并口）这种帧式接口实现不了**。
- 帧级：要求 BSP 自己判帧边界，等于把组帧搬进 BSP，**违反「BSP 不解析」的既有规则**。
- 「字节块」是两者的交集：UART 实现投递 1 字节或一小段，DCMI 实现投递一整帧，Service 的组帧状态机对两种粒度都按字节消费。

两种物理层如何映射到同一端口

端口函数	`_uart.c`（相机在独立算力板）	`_dcmi.c`（相机直挂 MCU）
`Init`	打开串口 + DMA 接收 + 空闲中断	配 DCMI + DMA 双缓冲
收数据	中断/DMA 收字节 → 投递	DMA 帧完成回调 → 整帧投递
`SendBuf`	真实发送	空实现（无下行链路）
`Trigger`	发取景命令	写 DCMI 触发位
`IsReady`	串口就绪 + 对端心跳	DCMI 已启动
`ReleaseFrame`	不需要	**需要**：归还 DMA 缓冲（若采用帧级投递则要补进端口）

Service 侧结果结构草案（`Service/Inc/cam_frame.h`）

```
typedef struct {
    s16 targetX;       /* 目标横向偏移（量纲由 cfg_cam.h 定义） */
    s16 targetY;
    u16 distanceMm;    /* 深度距离 */
    u8  valid;         /* 0 = 本帧无效 */
    u32 timestampMs;
} DepthCam_Result;

void Service_DepthCam_FeedBlock(const u8 *data, u16 len);  /* 注册给 BSP */
void Service_DepthCam_TakeResult(DepthCam_Result *out);    /* App 轮询 */
```

硬约束

- ⚠️ **深度图不上 MCU，只传结果/特征点**。主 RAM 只有 128 KB，放不下一张 QVGA 灰度图。

- ⚠️ 若真走 DCMI，只能行缓冲 + DMA，且缓冲区**必须在主 RAM，不能在 CCM**（CCM 不能被 DMA 访问，见 B6）。
- ⚠️ 相机断线/数据异常时上层必须有兜底（`IsReady() == 0` 或 `valid == 0` 时按「无目标」处理），不能卡死。
- 💡 先写 `bsp_depthcam_null.c`：今天就能把 C2/C5 写完跑通；等硬件确定，只加一个实现文件 + 改一行 CMake，**上层一行不动**。

9.7 路线三：加操作系统（RTOS）

目标：从裸机大循环换成 RTOS，同时不破坏 `Osal` 这层抽象。

编号	项	落点	动作	风险
R1	引入 FreeRTOS	`.ioc` 勾选 FreeRTOS（CMSIS-RTOS v2）	CubeMX 生成 `Core/Src/freertos.c`，并新增 `Middlewares/Third_Party/FreeRTOS/`	低
R2	**Osal 换实现**	`Osal/Src/osal.c`	`HAL_Delay` → `vTaskDelay(pdMS_TO_TICKS())`；`HAL_GetTick` → `xTaskGetTickCount()*portTICK_PERIOD_MS`	低（架构已预留）
R3	**Osal 是否长出 task/sync API**（见 D3）	新增 `Osal/Inc/osal_task.h`、`Osal/Inc/osal_sync.h` + 对应 `.c`（并注册进 `CMakeLists.txt` 的 `osal` 库）	若「长出来」，上层永远不 include `FreeRTOS.h`，防腐完整	架构决策
R4	**六段动作搬进任务**	`Core/Src/main.c` USER CODE 2 段 → 新建 App 任务	它们会调 `Osal_DelayMs` → `vTaskDelay`，**在 `osKernelStart()` 之前调用是非法的**	**高**
R5	`Motion_*` 阻塞 API 处理（见 D4）	`App/Inc/motion.h`、`App/Src/motion.c`	方案甲（保守）：运动独占一个高优先级任务，签名不变；方案乙（彻底）：改非阻塞状态机 `Motion_Tick()`	**中**（可能改变运动轨迹）
R6	栈与堆预算	`STM32F407xx_FLASH.ld`、新增 `cfg_rtos.h`	FreeRTOS heap 与任务栈从主 RAM 分；CCM 不能给 DMA；`_Min_Stack_Size` 现值 1 KB	中

有利条件

- ✅ **HAL 时基已经在 TIM6 上、`SysTick_Handler()` 是空的**（B4）。SysTick 可以干净地交给 FreeRTOS，`HAL_Delay` / `HAL_GetTick` 不受影响。这个工程提前做对了，RTOS 迁移比一般裸机工程顺畅。
- ✅ **`Osal` 已经把时间接口收口**，换实现只改 `osal.c` 四行。

三个必须提前处理的地雷

- ⚠️ **NVIC 分组**（B5）：`HAL_MspInit()` 把分组覆盖成 GROUP_2，只有 0~3 级抢占优先级；FreeRTOS 需要 GROUP_4，否则 `vPortValidateInterruptPriority()` 直接断言。**先做 X3**。
- ⚠️ **中断里调 RTOS API 必须用 FromISR 版本**：`Protocol_FeedByte()` 在中断上下文被调（契约 7）。若它内部要用队列/信号量把数据交给任务，必须用 `...FromISR()`。
- ⚠️ **中断优先级必须低于 `configMAX_SYSCALL_INTERRUPT_PRIORITY`**：USART1 当前抢占优先级是 3，改 GROUP_4 后要重新确认它「够低」到可以安全调用 RTOS API。

RAM 预算提醒

主 RAM 只有 128 KB（不含 CCM，CCM 又不能给 DMA）。FreeRTOS heap + 各任务栈都要从这里出。做法：先用 `.map` 记录当前占用，每加一个任务就回归一次，目标 **RAM 占用 < 90%**。

9.8 建议实施顺序与阶段验收

每阶段都能独立验收，不必一次做完。

阶段	内容	为什么排这里	独立验收标准
0	§9.4 的 X1~X6（必修缺陷 + 配置头）	风险最低、收益最高；后面三条路线都依赖它	编译通过；串口压测 10 万帧不失聪；故障有输出；`.map`/`.su` 正常
1	§9.5 上位机通信（H1~H5）	可在**裸机上完整验收**，不引入 RTOS，先把手写契约焊死	PC 端脚本下发命令 → 电机按预期动作；上位机拿到状态上报；连发不丢帧；`docs/protocol.md` 可让第三方独立实现
2	§9.6 相机端口预留（C1~C4）	端口先定型，之后接 RTOS 时接口**不会被 RTOS 的形状污染**	`null` 实现下工程编译运行、行为与改造前一致；切换物理层只需改 CMake，不动上层代码

阶段	内容	为什么排这里	独立验收标准
3	§9.7 RTOS 接入（R1~R6）	依赖阶段 0 的 X3；依赖阶段 2 已定型的端口	任务栈 `.su` 无溢出预警；`.map` RAM < 90%；上电六段动作在 RTOS 下复现（示波器 PE9：2500 Hz、周期 400 μs、高电平 200 μs）
4	相机真实实现 + C5（真正接入）	物理层此时才确定	端到端：相机 → App 决策的时延与掉帧率达标

> 阶段 2 与 3 可以互换（若更想先上 RTOS）。但**阶段 0 必须最先**。

逐阶段检查清单

- 阶段 0：☐ 编译通过（Debug / Release） ☐ 压测 10 万帧不失聪 ☐ 故障现场有输出 ☐ 双串口（若已加）各自正常
- 阶段 1：☐ 命令下发动作正确 ☐ 收到状态上报 ☐ 连发不丢帧 ☐ `docs/protocol.md` 可独立实现上位机
- 阶段 2：☐ `null` 实现下行为与改造前一致 ☐ 换物理层只改 CMake 一处
- 阶段 3：☐ 无栈溢出预警 ☐ `.map` RAM < 90% ☐ 六段动作在 RTOS 下复现 ☐ 高负载串口不失聪
- 阶段 4：☐ 端到端时延与掉帧率达标 ☐ 相机断线/异常不卡死

9.9 待拍板的决策点

开工前需要有人拍板这六条。**没定下来的，建议先按「保守选项」推进**（下面括号里标了）。

编号	决策	选项	影响范围
D1	深度相机最终物理接口	独立算力板串口 / DCMI 并口 / SPI	**只决定 C3 选哪个实现文件，端口与上层不受影响**。不确定就先 `_null.c` 占位
D2	RTOS 选型	FreeRTOS（CMSIS-RTOS v2） / RT-Thread / 暂不上 RTOS	只影响 `osal.c` 的实现内容。**推荐 FreeRTOS**（CubeMX 直接生成，与现工程集成最省事）
D3	`Osal` 是否「长出」 task/sync API	长（上层永不 include `FreeRTOS.h`，防腐完整） / 不长（只抽象时间，任务创建散落 App）	防腐规则的完整度。**推荐「长出来」**，否则 RTOS 会渗透到 Service/App
D4	`Motion_*` 重构方式	甲：运动独占任务（签名不变） / 乙：非阻塞状态机 `Motion_Tick()`	是否改变现有运动轨迹与时序。**建议先甲**，跑通再考虑乙
D5	上位机帧格式	文本行 + 超时（改动小） / 帧头 + 长度 + CRC16（更可靠）	上位机侧的开发量。**若能约束上位机代码，建议乙**（长跑可靠性差别很大）
D6	上位机是否走 USB CDC	是 / 否（继续用 USART1）	若选「是」，需引入 `Middlewares/ST/STM32_USB_Device_Library`（**当前工程没有 `Middlewares/` 目录**），并新增一个与 `Drivers/` 同级的底座目录

9.10 风险登记表

ID	风险	触发条件	缓解措施
RK1	改 NVIC 分组影响现有中断	做 X3 时	一次性重算 USART1 / TIM6 优先级；用示波器 + 串口回归
RK2	S 曲线重构改变运动轨迹	R5 选方案乙时	保留 `Motion_Move` 作为回归基准；用 §4.4 的示波器判据逐段比对
RK3	六段演示搬进任务后上电时序变化	R4	把「上电动作」做成一个任务的首段流程，复位行为可复现
RK4	相机端口抽象选错，需要返工	C1	返工被限制在 BSP 实现文件内；Service/App 不受影响（这正是端口分层的价值）
RK5	FreeRTOS heap 与 128 KB 主 RAM 冲突	R6	用 `.map` 与各函数 `.su` 报告核定；目标 RAM 占用 < 90%
RK6	相机整帧缓冲挤占 RAM	C2 / C5	只传结果不上图；DCMI 分支用行缓冲 + DMA，且禁止放 CCM
RK7	CubeMX 重新生成覆盖手写代码	任何阶段	严守 P1/P2；只写 `USER CODE` 段
RK8	中文路径导致命令行构建卡死	任何阶段	只用 IDE 的 Build / F5，不用命令行 `ninja`
RK9	改时钟树忘记同步常量 → 转速静默出错	R1（或任何改 `.ioc` 的操作）	每次动 `.ioc` 后核对 `cfg_motor.h` 的 `CFG_TIM1_CLK_HZ` / `CFG_TIM2_CLK_HZ`
RK10	新增 `.c` 忘记注册进 CMake，功能静默失效	任何阶段	遵守 P4；改完在 `.map` 里搜一下新符号

9.11 一句话交接总结

现有工程是一个分层干净、CubeMX 隔离良好的裸机四轮驱动底座：硬件知识全在 `Bsp`，OS 切换点全在 `Osal`，业务唯一入口是 `App_Init/App_Loop`。 **下一步不是重写，而是三件事按序叠加**：先修串口错误恢复等必修缺陷（阶段 0），再在裸机上把上位机双向协议做完（阶段 1），然后为相机预留物理层无关的端口（阶段 2），最后在这三块都定型之后接入 RTOS（阶段 3）。 **贯穿始终的三条纪律**：不碰 `Drivers/`、`Core/` 只写 `USER CODE` 段、新增源文件必须注册进 `CMakeLists.txt`。